Software Architecture. Object Oriented Design exercise:

A city wants to implement a new system for managing all the taxes relative to vehicles: cars and motorcycles. Vehicles are identified by a plate number plus a maker and model. Also, people are identified in the system by license number, but we also want to store its name, surname and address.

The system must be able to add, edit and delete both vehicles and persons. When a vehicle is sold to a person, the system must be able to calculate the annual taxes for this vehicle. The tax depends on the vehicle type. Motorcycles are taxed by a 10% of its engine displacement. Petrol cars are taxed by a 1,4 € per gram of CO_2 emissions, while diesel cars pay 1,8€ per gram of CO_2. Finally, hybrid cars only pay 1,2 € per gram of CO_2.

Once a year, the system must generate a list of all the vehicle owners, with the vehicles they have, the taxes for each vehicle and the total for each owner. It can happen that a person does not have any vehicle in a given moment and that a car is still in the car dealership so regarding tax point of view, it does not have an owner. Another case that can happen is that a vehicle owned by a person is sold or transferred to a different person.

Finally, the system also must be able to search for users and vehicles.
For this exercise we require the following results:

1. Use cases description and diagram.
2. Class diagram.
3. Sequence diagrams for the transfer vehicle and taxes listing use cases.
4. Implementation in Java.

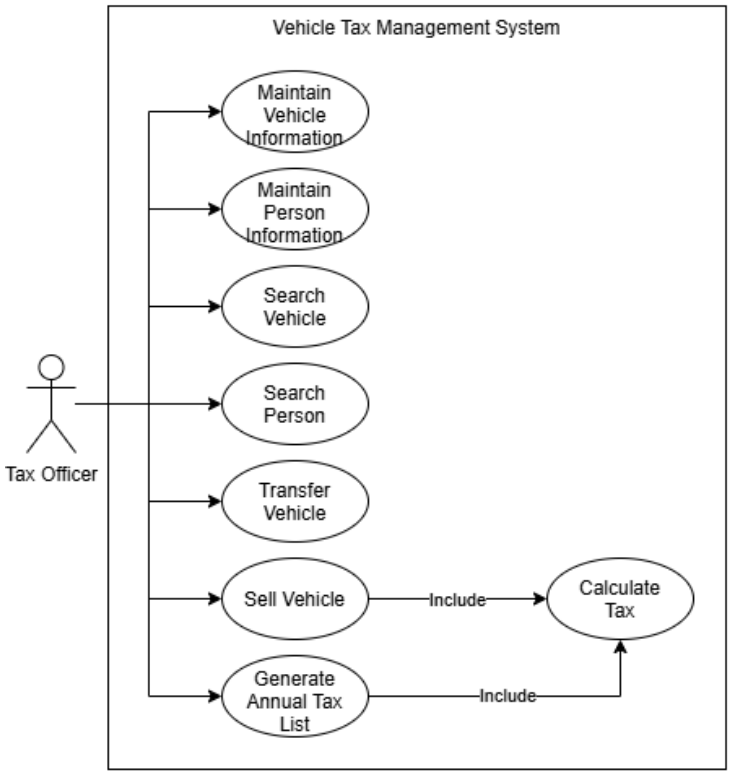
1. USE CASES DESCRIPTION AND DIAGRAM

To identify the use cases of the system, the first step is to identify its actors. In this system there is only one actor, the Tax Officer that is the city hall employee who registers vehicles and persons and manages the vehicle tax process. The persons described in the statement (users or vehicle owners) are not actors, since they do not interact with the system directly.

Once this has been found, the uses cases found were:

- Maintain vehicle information (add, edit, delete)
- Maintain user information (add, edit, delete)
- Search vehicle
- Search user
- Sell vehicle, include calculate tax
- Transfer vehicle
- Generate annual tax list, include calculate tax
- Calculate tax

The diagram for them:



Calculate Tax is not started by the Tax Officer. It is included (<<include>>) by Sell Vehicle and Generate Annual Tax List, because both always need to calculate the annual tax of a vehicle. Factoring it out into its own use case avoids duplicating this behaviour. Transfer vehicle does not include it because the tax depends only on the vehicle's characteristics (Engine displacement or CO_2 emissions and fuel type), not on its owner, so it does not change when the vehicle is transferred.

The add, edit and delete operations have been grouped into a single Maintain use case for vehicles and another one for persons/users. Each operation is described as an alternative flow in the corresponding use case description.

More details are provided for each one of the cases:

Vehicle information (add, edit, delete)	**Use Case:** Maintain Vehicle Information **ID:** UC-01 **Brief Description:** The Tax Officer adds, edits or deletes a vehicle (car or motorcycle) in the system. **Primary Actor:** Tax Officer **Pre-Conditions:** None. **Minimum Guarantees**: If the operation fails, the stored vehicle data is not modified and an error message is shown. **Success Guarantees**: The vehicle has been added, updated or deleted. **Primary Flow of Events:** 1. The Tax Officer selects the "Maintain vehicles" option. 2. The System offers the following options: Add a vehicle, Edit a vehicle, Delete a vehicle. 3. The Tax Officer selects an option and completes the task. 4. The System saves the changes and shows a confirmation. 5. The use case ends. **Alternative Flow of Events:** **3a:** Tax Officer selects to Add a vehicle 1. The System asks for the plate number, maker, model and vehicle type (motorcycle or car). 2. The Tax Officer enters the data. 3. If the vehicle is a motorcycle, the System asks for the engine displacement. If it is a car, the System asks for the fuel type (petrol, diesel or hybrid) and the CO2 emissions (g). 4. The Tax Officer enters the data. 5. The System checks that the plate number does not already exist and creates the vehicle with no owner. 6. The use case resumes at step 4 (of Primary flow). **3a.5a:** Plate number already exists 1. The System shows an error stating that the vehicle is already registered. 2. The use case resumes at step 1 (of Alternative flow 3a). **3a.5b:** Missing or invalid data 1. The System shows an error indicating the invalid fields. 2. The use case resumes at step 1 (of Alternative flow 3a). **3b:** Tax Officer selects to Edit a vehicle 1. The System asks for the plate number. 2. The Tax Officer enters the plate number. 3. The System shows the vehicle's data. 4. The Tax Officer modifies the data (except the plate number, which identifies the vehicle). 5. The use case resumes at step 4 (of Primary flow). **3b.3a:** Vehicle not found 1. The System shows an error message. 2. The use case resumes at step 1 (of Alternative flow 3b). **3c:** Tax Officer selects to Delete a vehicle 1. The System asks for the plate number. 2. The Tax Officer enters the plate number. 3. The System shows the vehicle's data and asks for confirmation. 4. The Tax Officer confirms. 5. The System deletes the vehicle and, if it had an owner, removes it from the owner's vehicles. 6. The use case resumes at step 4 (of Primary flow).

	3c.3a: Vehicle not found 1. The System shows an error message. 2. The use case resumes at step 1 (of Alternative flow 3c). **3c.4a:** Tax Officer cancels the deletion 1. The use case ends without changes.
User information (add, edit, delete)	**Use Case:** Maintain User Information **ID:** UC-02 **Brief Description:** The Tax Officer adds, edits or deletes a user in the system. **Primary Actor:** Tax Officer **Pre-Conditions:** None. **Minimum Guarantees:** If the operation fails, the stored user data is not modified and an error message is shown. **Success Guarantees:** The user has been added, updated or deleted. **Primary Flow of Events:** 1. The Tax Officer selects the "Maintain user" option. 2. The System offers the following options: Add a user, Edit a user, Delete a user. 3. The Tax Officer selects an option and completes the task. 4. The System saves the changes and shows a confirmation. 5. The use case ends. **Alternative Flow of Events:** **3a:** Tax Officer selects to Add a user 1. The System asks for the license number, name, surname and address. 2. The Tax Officer enters the data. 3. The System checks that the license number does not already exist and creates the user with no vehicles. 4. The use case resumes at step 4 (of Primary flow). **3a.3a:** License number already exists 1. The System shows an error stating that the user is already registered. 2. The use case resumes at step 1 (of Alternative flow 3a). **3a.3b:** Missing or invalid data 1. The System shows an error indicating the invalid fields. 2. The use case resumes at step 1 (of Alternative flow 3a). **3b:** Tax Officer selects to Edit a user 1. The System asks for the license number. 2. The Tax Officer enters the license number. 3. The System shows the user's data. 4. The Tax Officer modifies the name, surname or address. 5. The use case resumes at step 4 (of Primary flow). **3b.3a:** User not found 1. The System shows an error message. 2. The use case resumes at step 1 (of Alternative flow 3b). **3c:** Tax Officer selects to Delete a user 1. The System asks for the license number. 2. The Tax Officer enters the license number. 3. The System shows the user's data and asks for confirmation. 4. The Tax Officer confirms. 5. The System deletes the user. 6. The use case resumes at step 4 (of Primary flow). **3c.3a:** User not found 1. The System shows an error message. 2. The use case resumes at step 1 (of Alternative flow 3c). **3c.3b:** User owns vehicles

	1. The System warns that the user owns vehicles and that they will be left with no owner. 2. The use case resumes at step 4 (of Alternative flow 3c). **3c.4a:** Tax Officer cancels the deletion 1. The use case ends without changes.
Search vehicle	**Use Case:** Search Vehicle **ID:** UC-03 **Brief Description:** The Tax Officer searches for a vehicle by its plate number and consults its data and its current owner. **Primary Actor:** Tax Officer **Pre-Conditions**: None. **Minimum Guarantees:** No data is modified. **Success Guarantees:** The vehicle's data and its owner (if any) are shown. **Primary Flow of Events:** 1. The Tax Officer selects the "Search vehicle" option. 2. The System asks for the plate number. 3. The Tax Officer enters the plate number. 4. The System finds the vehicle and shows its plate number, maker, model, type, type-specific data and current owner. 5. The use case ends. **Alternative Flow of Events:** **4a:** Vehicle not found 1. The System shows a message stating that no vehicle was found. 2. The use case resumes at step 2. **4b:** Vehicle has no owner 1. The System shows the vehicle's data and indicates that it has no owner. 2. The use case ends.
Search user	**Use Case:** Search user **ID:** UC-04 **Brief Description:** The Tax Officer searches for a user by their license number and consults their data and the vehicles they own. **Primary Actor:** Tax Officer **Pre-Conditions:** None. **Minimum Guarantees:** No data is modified. **Success Guarantees:** User's data and their vehicles (if any) are shown. **Primary Flow of Events:** 1. The Tax Officer selects the "Search user" option. 2. The System asks for the license number. 3. The Tax Officer enters the license number. 4. The System finds the user and shows their license number, name, surname, address and the list of vehicles they own. 5. The use case ends. **Alternative Flow of Events:** **4a:** User not found 1. The System shows a message stating that no user was found. 2. The use case resumes at step 2. **4b:** User has no vehicles 1. The System shows the user's data and indicates that they own no vehicles. 2. The use case ends.

Sell vehicle, include calculate tax	**Use Case:** Sell Vehicle **ID:** UC-05 **Brief Description:** The Tax Officer registers the sale of a vehicle that has no owner (e.g. still at the dealership) to a user. The system assigns the owner and calculates the vehicle's annual tax. **Primary Actor:** Tax Officer **Pre-Conditions:** 1. The vehicle and the buyer are registered in the system. 2. The vehicle has no owner. **Minimum Guarantees:** If the sale fails, the vehicle's ownership is not modified and an error message is shown. **Success Guarantees:** 1. The vehicle is assigned to the buyer. 2. The annual tax for the vehicle has been calculated. **Primary Flow of Events:** 1. The Tax Officer selects the "Sell vehicle" option. 2. The System asks for the vehicle's plate number. 3. The Tax Officer enters the plate number. 4. The System finds the vehicle and checks that it has no owner. 5. The System asks for the buyer's license number. 6. The Tax Officer enters the license number. 7. The System finds the user and assigns the vehicle to them. 8. The System calculates the annual tax (include: Calculate Tax). 9. The System shows a confirmation with the tax amount. 10. The use case ends. **Alternative Flow of Events:** **4a:** Vehicle not found 1. The System shows an error message. 2. The use case resumes at step 2. **4b:** Vehicle already has an owner 1. The System informs that the vehicle already has an owner and that "Transfer Vehicle" must be used instead. 2. The use case ends. **7a:** User not found 1. The System shows an error message. 2. The use case resumes at step 5.
Transfer vehicle	**Use Case:** Transfer Vehicle **ID:** UC-06 **Brief Description:** The Tax Officer registers the transfer of a vehicle from its current owner to a new owner. The annual tax is not recalculated, because it depends only on the vehicle's characteristics, not on its owner. **Primary Actor:** Tax Officer **Pre-Conditions:** 1. The vehicle, the current owner and the new owner are registered in the system. 2. The vehicle has an owner. **Minimum Guarantees:** If the transference fails, the vehicle's ownership is not modified, and an error message is shown. **Success Guarantees:** 1. The vehicle is assigned to the new owner. 2. The vehicle is removed from the current owner’s possession. **Primary Flow of Events:** 1. The Tax Officer selects the "Transfer vehicle" option.

	2. The System asks for the vehicle's plate number. 3. The Tax Officer enters the plate number. 4. The System finds the vehicle and checks that it has an owner. 5. The System asks for the new owner's license number. 6. The Tax Officer enters the license number. 7. The System finds the new owner. 8. The System removes the vehicle from the current owner and assigns it to the new owner. 9. The System shows confirmation of the transfer. 10. The use case ends. **Alternative Flow of Events:** **4a:** Vehicle not found 1. The System shows an error message. 2. The use case resumes at step 2. **4b:** Vehicle has no owner 1. The System informs that the vehicle already has no owner and that "Sell Vehicle" must be used instead. 2. The use case ends. **7a:** User not found 1. The System shows an error message. 2. The use case resumes at step 5. **7b:** New owner is the same as the current owner 1. The System shows an error message. 2. The use case resumes at step 5.
Generate annual tax list, include calculate tax	**Use Case:** Generate Annual Tax List **ID:** UC-07 **Brief Description:** Once a year, the Tax Officer generates a list of all vehicle owners with the vehicles they own, the tax for each vehicle and the total tax for each owner. **Primary Actor:** Tax Officer **Pre-Conditions:** None. **Minimum Guarantees:** No data is modified. **Success Guarantees:** The list of all vehicle owners with their vehicles, the tax for each vehicle and the total per owner is shown. **Primary Flow of Events:** 1. The Tax Officer selects the "Generate annual tax list" option. 2. The System retrieves all the users who own at least one vehicle. 3. For each owner, the System: Retrieves the vehicles they own; Calculates the tax for each vehicle; (include: Calculate Tax); Adds up the total tax for the owner. 4. The System shows the list with, for each owner: license number, name and surname, each vehicle (plate number, maker, model and tax) and the total tax. 5. The use case ends. **Alternative Flow of Events:** **2a:** There are no vehicle owners 1. The System shows a message stating that there are no vehicle owners. 2. The use case ends. **Notes:** Users without vehicles are not included in the list, since they are not vehicle owners. Vehicles without an owner (e.g. still at the dealership) are not included either, since nobody pays their tax.
Calculate tax	**Use Case:** Calculate Tax **ID:** UC-08

	Brief Description: The System calculates the annual tax of a vehicle according to its type. This use case is not started by an actor: it is included by “Sell Vehicle" and "Generate Annual Tax List". **Primary Actor:** None. **Pre-Conditions:** The vehicle is registered in the system with all the data needed for its type. **Minimum Guarantees:** No data is modified. **Success Guarantees:** The annual tax of the vehicle is returned to the including use case. **Primary Flow of Events:** 1. The including use case provides the vehicle. 2. The System checks the vehicle type. 3. The System calculates the tax: Motorcycle: 10% of the engine displacement. Petrol car: 1.4 € per gram of CO2 emissions. Diesel car: 1.8 € per gram of CO2 emissions. Hybrid car: 1.2 € per gram of CO2 emissions. 4. The System returns the tax to the including use case. 5. The use case ends. **Alternative Flow of Events:** None.

2. CLASS DIAGRAM

The classes that have been found are the following: *TaxSystem, Person, Vehicle, Car, Motorcycle* and *FuelType*. They were identified by analyzing the nouns in the statement (vehicle, car, motorcycle, person, city, tax, officer, plate number…) and filtering them: a noun becomes a class only if the system needs to store data about it or if it has its own behavior. The rest of the nouns were treated in different ways: some became attributes (plate number, maker, model, engine displacement, CO_2 emissions), the tax became a method (calculateTax()), and others were discarded, such as city, which is only the context of the system, and Tax Officer, which is the actor and therefore lies outside the system. The *TaxSystem* class does not appear in the statement, but it is needed as the entry point of the system: it stores all the persons and vehicles and offers one method for each use case.

Cars and motorcycles share the attributes plate number, maker and model, and both must calculate an annual tax. For this reason, a generalization has been used: Car and Motorcycle inherit from Vehicle which contains the common attributes, and each subclass only adds its own data. Vehicle is an abstract class, since every vehicle in the system is either a car or a motorcycle, and a generic vehicle can’t have a tax.

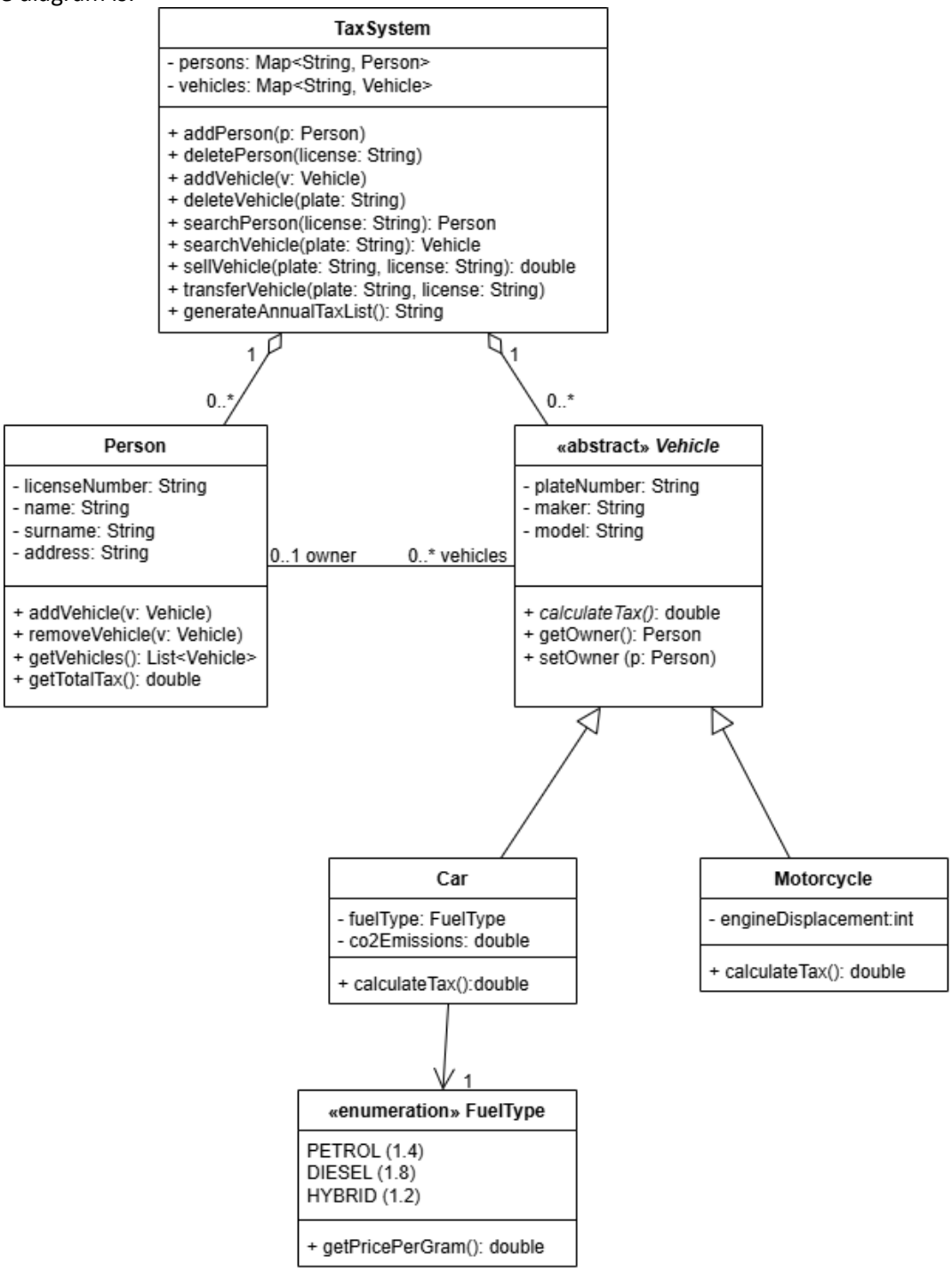
The method calculateTax() is abstract in Vehicle and is overridden in each subclass with its own formula. It has been placed in the vehicle. Thanks to polymorphism, the rest of the system calls calculateTax() without knowing the type of each vehicle. For example, when generating the annual tax list each vehicle calculates its own tax.

Petrol, diesel and hybrid cars have the same attributes and the same behaviour, they only differ in the price per gram of CO_2. Since there is not different data or behavior, an enumeration FuelType has been used instead, where each value stores its price per gram.

Regarding the relationships and multiplicities, Person and Vehicle are related by an association with multiplicities 0…1 and 0…*: a vehicle has 0 or 1 owner and a person owns 0 or more

vehicles. The zero on each side makes references to when a vehicle is still at the car dealership and has no owner and when a person may own no vehicles at a given moment. TaxSystem is related to Person and Vehicle by aggregation, since it contains the collections of all the persons and vehicles registered in the system.

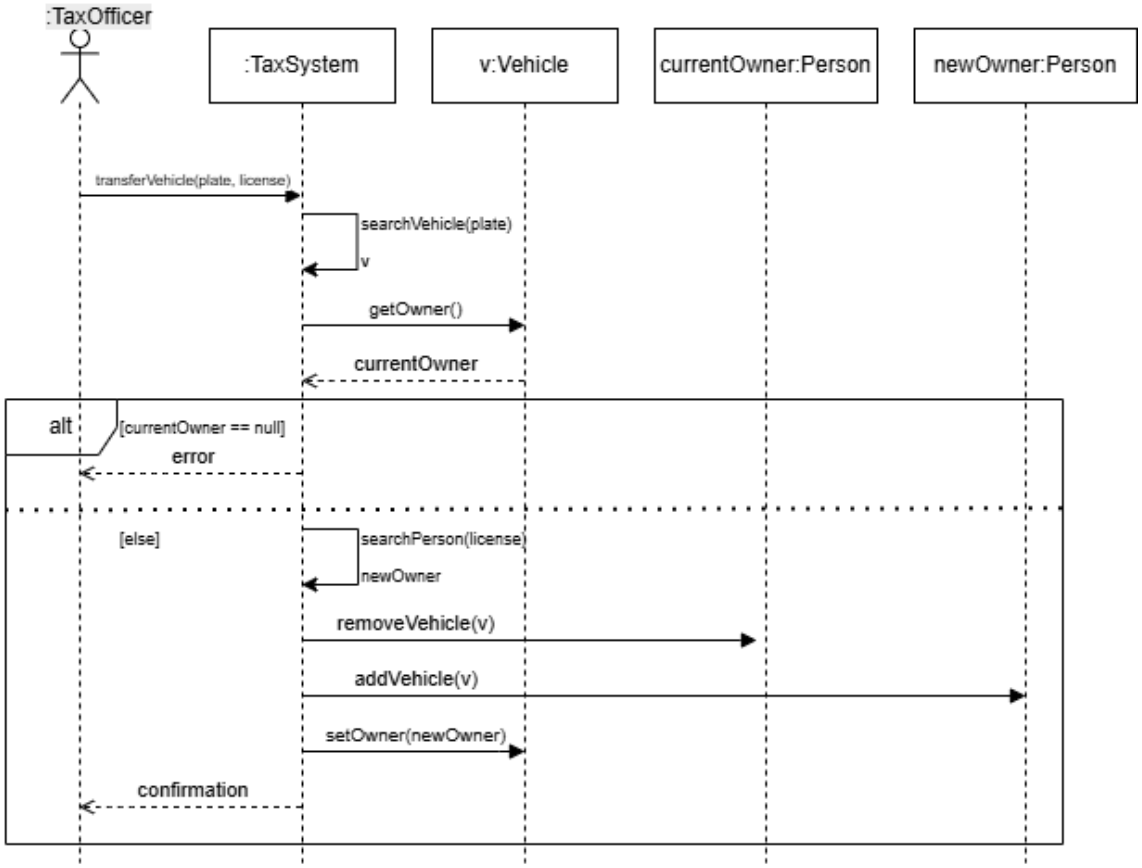
The diagram is:



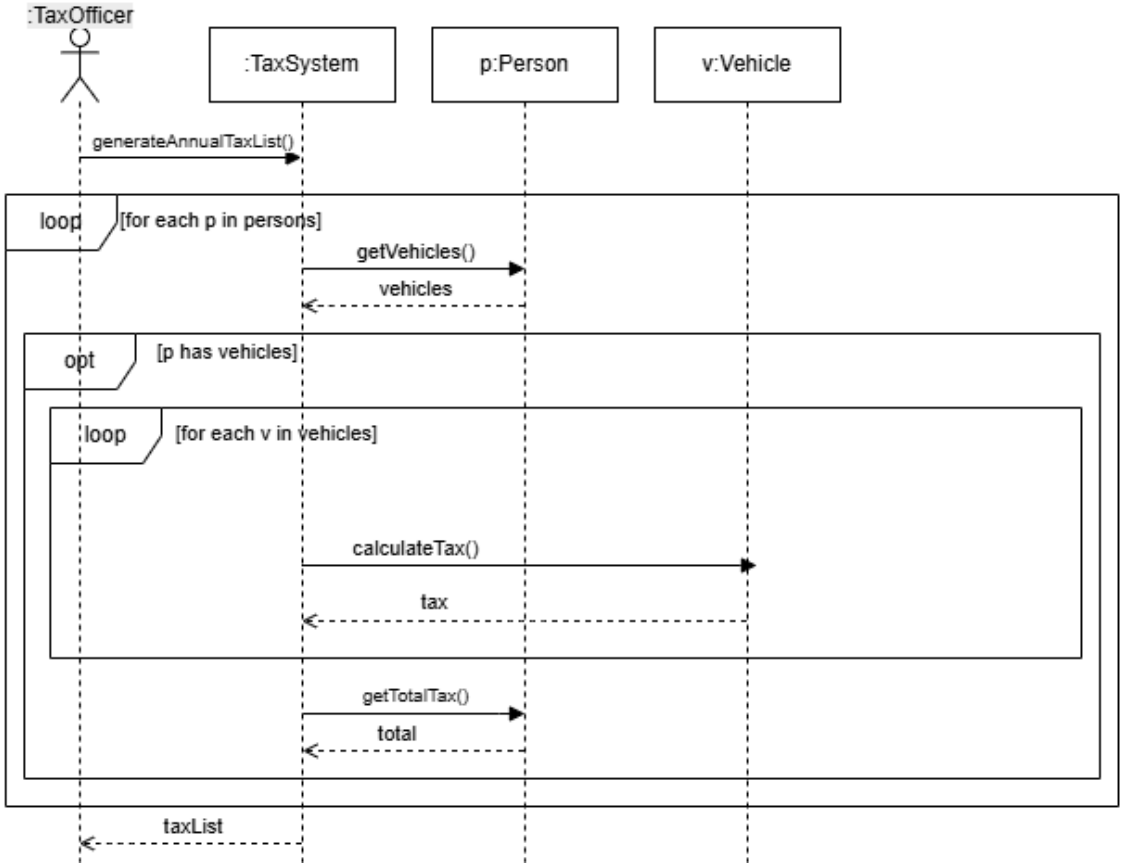
3. SEQUENCE DIAGRAMS FOR THE TRANSFER VEHICLE AND TAXES LISTING USE CASES.

Sequence diagrams show how the objects of the system interact over time to carry out a use case. Each lifeline represents an object, and each message corresponds to a method call.

The Transfer vehicle diagram:



The Taxes Listing diagram:



4. IMPLEMENTATION IN JAVA. YOU CAN USE THE JAVA.UTIL CLASSES THAT YOU NEED.

The complete source code of the implementation is available in the following GitHub repository: https://github.com/marionamontalmorta/OOD-FourthAssignmentNETENG .